

Subject: SAST Summer Training 2026, Agent Harness

Date: from 2026/10/11 to 2026/10/12

Contents

Query Loop / Agent Loop	1
--------------------------------	----------

Query Loop / Agent Loop

循环总览

Query Loop 是 Harness 的核心状态机. 每一轮都会先整理上下文, 再调用模型; 如果模型请求工具, 就执行工具并把结果加入下一轮, 否则结束当前任务.

5. queryLoop()

- 5.1 创建动态 State
 - messages
 - toolUseContext
 - turnCount
 - Compact 状态
 - Token 恢复计数
- 5.2 检查上下文容量
 - 估算当前 token
 - 判断是否需要 auto compact
 - compactConversation()
 - buildPostCompactMessages()
- 5.3 准备模型请求
 - system prompt
 - CLAUDE.md / 项目上下文
 - conversation messages
 - Skill / Hook 注入内容
 - Tool Schema
 - normalizeMessagesForAPI()
- 5.4 调用模型并消费流式响应
 - text delta
 - thinking / signature 等协议块
 - tool_use block
 - usage / stop_reason
 - 生成 assistant message
- 5.5 判断是否需要执行工具
 - 没有 tool_use
 - 保存最终 assistant message
 - 更新 session / telemetry
 - 结束 queryLoop
 - 存在 tool_use
 - 根据 name 查找 Tool
 - 校验 input schema
 - checkPermissions()
 - allow / deny / 用户确认
 - 执行 Tool.call()
 - 收集 progress 和最终结果
- 5.6 将工具结果转成消息
 - 收集 stdout / stderr / exit code
 - 截断或持久化超大结果
 - 创建 tool_result block

```

|   | - 设置 tool_use_id
|   | - 包装成下一轮 user message
|
| - 5.7 更新状态
|   | - 追加 assistant tool_use message
|   | - 追加 user tool_result message
|   | - 写入 transcript
|   | - 更新 telemetry
|   | - turnCount + 1
|
| - 5.8 返回循环顶部
|   | - 模型根据 tool_result 决定下一步

```

关键控制流

下面的缩略版代码展示关键控制流. 编号与上方流程图对应, 部分函数是为了讲解而保留的抽象名称.

```

// query() 是一次任务的外层包装器: 它转发 queryLoop() 产生的全部事件,
// 并只在循环正常结束时提交命令生命周期状态.
export async function* query(params: QueryParams) {
  // 记录本次任务实际消费过的命令, 便于结束时统一标记完成.
  const consumedCommandUids: string[] = []

  // yield* 会把内部生成器的事件原样向上传递, 并接收其最终 return 值.
  const terminal = yield* queryLoop(params, consumedCommandUids)

  // 只有 queryLoop 正常返回才记录 completed.
  for (const uuid of consumedCommandUids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}

// queryLoop() 持续维护动态状态, 直到模型给出不含工具调用的最终回答,
// 或任务因取消、错误、轮数限制等条件提前结束.
async function* queryLoop(params: QueryParams, consumedCommandUids:
string[]) {
  // 生产环境使用真实依赖; 测试可以通过 params.deps 注入替身实现.
  const deps = params.deps ?? productionDeps()

  // 权限回调在每次工具执行前决定 allow、deny 或 ask.
  const { canUseTool } = params

  // State 保存跨轮次变化的数据. 它与只描述初始输入的 QueryParams 不同.
  let state: State = {
    messages: params.messages,
    toolUseContext: params.toolUseContext,
    maxOutputTokensOverride: params.maxOutputTokensOverride,
    autoCompactTracking: undefined,
    maxOutputTokensRecoveryCount: 0,
    hasAttemptedReactiveCompact: false,
    turnCount: 1,
  }
}

```

```

// 每次迭代代表一次“构建上下文 -> 调用模型 -> 处理工具”的 Agent Turn.
while (true) {
  // 使用局部变量完成本轮处理，轮末再一次性写回 state.
  let { messages, toolUseContext } = state

  // 每轮开始做 checkpoint；源码还会应用 tool-result budget、history snip、
  // microcompact 和 context collapse。这些是请求前的准备，不是一个单独的
  // `prepareContext()` 函数。
  queryCheckpoint('query_fn_entry')

  // 1. 自动 Compact 是请求模型前的容量检查，不是循环结束。真实代码先后
  // 处理 applyToolResultBudget / snip / microcompact / contextCollapse，
  // 再调用
  // deps.autocompact()；这里将前置准备折叠成注释。
  const { compactionResult } = await deps.autocompact(
    messages,
    toolUseContext,
    /* context, querySource, tracking ... */
  )

  // 发生压缩时，用摘要和必要的边界消息替换旧历史；否则保留原消息。
  messages = compactionResult
    ? buildPostCompactMessages(compactionResult)
    : messages

  // 2. deps.callModel 会 yield StreamEvent 和组装完成的 AssistantMessage。
  // 原始 content_block_delta 的拼装在 services/api/claude.ts，不在 query.ts。
  // 收集本轮完整的 assistant 消息，供持久化和下一轮上下文使用。
  const assistantMessages: AssistantMessage[] = []

  // 单独收集 tool_use block，用它判断本轮是否需要进入工具执行阶段。
  const toolUseBlocks: ToolUseBlock[] = []

  // 工具结果在协议中以 user 消息承载，随后与 assistant 消息一起回填。
  const toolResults: UserMessage[] = []

  // 执行器负责工具查找、权限判断、并发调度和进度输出。
  const executor = new StreamingToolExecutor(
    toolUseContext.options.tools,
    canUseTool,
    toolUseContext,
  )

  try {
    // 源码通过 deps.callModel(...) 获得 AsyncGenerator；底层 API client 被
    // 依赖对象封装，queryLoop 不直接调用 `apiClient.messages.stream()`。
    for await (const message of deps.callModel({
      messages,
      tools: toolUseContext.options.tools,
      /* systemPrompt, model, signal, fallbackModel, ... */
    })) {
      // 先把流式消息交给上层 UI，保证展示不被后续解析阻塞。
      yield message

      if (message.type === 'assistant') {
        // 保存已完成组装的 assistant 消息。
        assistantMessages.push(message)
      }
    }
  }
}

```

```

    // 从消息内容中提取模型请求的所有 tool_use block.
    const blocks = message.message.content.filter(
      block => block.type === 'tool_use',
    ) as ToolUseBlock[]
    toolUseBlocks.push(...blocks)

    // 工具一旦被解析完整即可入队，允许其与剩余模型流并行执行.
    for (const block of blocks) executor.addTool(block, message)
  }

  // 已经执行完成的并行工具不必等模型流完全结束.
  for (const result of executor.getCompletedResults()) {
    if (result.message) {
      // 工具结果同样作为 Query 事件向外输出.
      yield result.message

      // 将内部结果规范化为模型 API 接受的 user/tool_result 消息.
      toolResults.push(...normalizeMessagesForAPI(
        [result.message],
        toolUseContext.options.tools,
      ).filter(m => m.type === 'user'))
    }
  }
} catch (error) {
  // query.ts 显式处理 fallback 和异常消息；普通网络重试由 deps.callModel
  // 及其 API 层负责。模型错误路径会调用真实的
  // yieldMissingToolResultBlocks(assistantMessages, errorMessage).
  if (isAbortError(error)) {
    // Anthropic 协议要求每个 tool_use 都有对应 tool_result；取消任务时也要
    // 为未完成调用生成中断结果，保持消息结构合法.
    yield* yieldMissingToolResultBlocks(assistantMessages, 'Interrupted
    by user')
    return buildInterruptedTerminal(state)
  }

  // 非取消类异常交给外层统一记录和展示，避免在这里静默吞掉.
  throw error
}

// 6. 不能只看 stop_reason；没有实际 tool_use 才能结束.
if (toolUseBlocks.length === 0) {
  // 真实源码：先执行 executePostSamplingHooks()，再经过 handleStopHooks()
  // 判断是否需要继续；正常完成时由 query() 返回 Terminal.transcript、
  // analytics 和 UI 事件在外围消费者/服务中记录，这里不虚构 persistTurn.
  void executePostSamplingHooks(assistantMessages, toolUseContext)

  // 返回最终 Terminal 会结束生成器，同时由外层 query() 接收该返回值.
  return finishAssistantTurn(assistantMessages)
}

// 7. StreamingToolExecutor 负责多工具并行、权限、进度和最终结果.
// 不启用 streaming executor 时，源码走 runTools(toolUseBlocks, ...).
try {
  for await (const event of executor.getRemainingResults()) {
    // permission_request / tool_progress / tool_result 都向外 yield.
    yield event
  }
}

```

```

        if (event.type === 'tool_result') {
            // deny、取消和进程非零退出都被包装成 observation,
            // 不会自动让 queryLoop 结束.
            toolResults.push(toToolResultMessage(event))
        }
    }
} catch (error) {
    if (isAbortError(error)) {
        // 中断时为已出现但未完成的 tool_use 补齐结果, 避免 orphan blocks.
        yield* yieldMissingToolResultBlocks(assistantMessages, 'Interrupted
        by user')
        return buildInterruptedTerminal(state)
    }
    // 工具层其他异常仍向外抛出, 由任务级错误处理器决定是否恢复.
    throw error
}

// 8. assistant 的 tool_use 和 Harness 生成的 tool_result 一起进入下一轮.
// 源码在此处还会追加 queued command、memory/skill/file-change attachments,
// 刷新 MCP tools, 并检查 maxTurns; 最终才构造下一轮 State. transcript、
// telemetry 和 Hook 由 query.ts 的真实调用点及外围 session/analytics
// 模块完成.
state = {
    // 保留没有在本轮显式修改的状态字段.
    ...state,

    // 严格保持协议顺序: 原历史 -> assistant/tool_use -> user/tool_result.
    messages: [...messages, ...assistantMessages, ...toolResults],
    toolUseContext,

    // 下一轮编号递增; 恢复计数只描述当前轮, 因此在成功推进后清零.
    turnCount: state.turnCount + 1,
    maxOutputTokensRecoveryCount: 0,
    hasAttemptedReactiveCompact: false,
}
}
}
}

```

QueryLoopState

QueryLoopState 保存 Agent Loop 在多轮执行中不断变化的状态:

```

type QueryLoopState = {
    // 当前完整消息历史; 每轮末尾都会追加模型消息和工具结果.
    messages: Message[]

    // 当前工具环境; 工具可以通过 contextModifier 返回更新后的上下文.
    toolUseContext: ToolUseContext

    // 自动压缩过程的跟踪信息, 例如压缩前后的 Token 估算.
    autoCompactTracking?: AutoCompactTrackingState

    // 输出超过限制后的恢复尝试次数, 用于避免无限重试.
    maxOutputTokensRecoveryCount: number
}

```

```
// 标记是否已经尝试过由错误触发的响应式压缩.  
hasAttemptedReactiveCompact: boolean  
  
// 当前生效的最大输出 Token 覆盖值.  
maxOutputTokensOverride?: number  
  
// 后台生成中的工具调用摘要; 尚未完成时表现为 Promise.  
pendingToolUseSummary?: Promise<ToolUseSummaryMessage | null>  
  
// 标记停止 Hook 是否正在运行, 防止重复进入.  
stopHookActive?: boolean  
  
// 当前 Agent Turn 编号, 通常从 1 开始递增.  
turnCount: number  
  
// 某些分支要求循环继续时携带的显式状态转移指令.  
transition?: Continue  
}
```

QueryParams 与 QueryLoopState 的区别

QueryParams 和 QueryLoopState 的区别是: 前者描述任务开始时拿到了什么, 后者描述任务执行到哪里、发生过什么.

QueryParams = 任务开始时拿到了什么
QueryLoopState = 任务执行到哪里、发生过什么